

HelixTrack

HelixTrack

JIRA alternativa para el mundo libre.

Resumen

HelixTrack es una alternativa moderna, integral y de código abierto a JIRA (y, mediante su extensión Documents, a Confluence), un sistema multiplataforma de gestión de proyectos y seguimiento de incidencias construido sobre un backend de microservicios Go con clientes nativos para web, escritorio y dispositivos móviles.

Descripción breve

Alternativa de código abierto a JIRA/Confluence. Un backend de microservicios Go («HelixTrack Core») expone una REST API unificada para la gestión de proyectos y el seguimiento de incidencias, además de un espacio de trabajo documental al estilo Confluence, servido a clientes nativos web, de escritorio, Android e iOS a través de HTTP/3 QUIC.

Descripción detallada

HelixTrack es una plataforma de gestión de proyectos y seguimiento de incidencias de código abierto, concebida como alternativa del mundo libre a JIRA y Confluence —un reemplazo completo de los dos productos a los que la mayoría de organizaciones de ingeniería están atadas, reconstruidos como software que posees y puedes ejecutar en cualquier lugar—. Su núcleo es **HelixTrack Core**, un microservicio REST API desarrollado en Go con el framework Gin, que ofrece seguimiento completo de incidencias, tableros ágiles/scrum, gestión de equipos y un motor de permisos jerárquicos cuya implementación es intercambiable entre un motor local en proceso y un servicio respaldado por HTTP. Esto permite que el mismo modelo de

autorización escale desde un solo portátil hasta un clúster distribuido sin modificar el código de la aplicación. En lugar de dispersar una superficie REST en docenas de rutas, Core canaliza todo a través de un único punto final /do con enrutamiento por acciones, utilizando un formato de solicitud/respuesta consistente (action/jwt/object/data de entrada, errorCode/errorMessage/data de salida): todos los clientes manejan el mismo contrato mínimo, y añadir una funcionalidad implica agregar una acción, no un nuevo URL que documentar, proteger y versionar. Core se integra con servicios desacoplados de Autenticación, Permisos y Localización que se comunican mediante HTTP/3 QUIC y pueden ejecutarse en máquinas o clústeres separados, o desactivarse por completo en configuraciones de prueba. Los datos se almacenan en SQLite para desarrollo sin configuración previa y en PostgreSQL en producción, cifrados en reposo con SQLCipher (AES-256), de modo que la información sensible de los proyectos queda protegida en disco por defecto, no como una ocurrencia tardía. La extensión **Documents V2** convierte el rastreador en una plataforma de conocimiento completa: un espacio de trabajo al estilo Confluence con espacios, páginas, control de versiones, plantillas, colaboración en tiempo real WebSocket y analíticas —la wiki y el rastreador de incidencias, por fin, conviviendo en un mismo backend en lugar de como dos productos unidos artificialmente—. Alrededor de Core se articulan múltiples aplicaciones cliente: un cliente web Angular, un cliente de escritorio Tauri + Angular, aplicaciones nativas para Android (Kotlin) e iOS (Swift), además de clientes para HarmonyOS y Aurora OS, e incluso un salvapantallas. Todas se comunican con el mismo backend y lo descubren automáticamente en redes locales mediante difusión UDP, de manera que un cliente recién instalado localiza su servidor sin configuración manual. Las aplicaciones cliente se mantienen en repositorios privados independientes y solo se presentan aquí a nivel de producto.

Por qué lo creamos

Para ofrecer a los equipos una alternativa genuinamente abierta y autoalojable al stack JIRA + Confluence —“para el mundo libre”— sin dependencia de proveedores, combinando seguimiento empresarial, documentos y colaboración bajo una única licencia de código abierto.

Por qué cambia las reglas del juego

Consolida dos productos comerciales de gran envergadura —el seguimiento de incidencias y el stack de wikis/documentos— en una plataforma abierta, de alto rendimiento y autoalojable, y lo acompaña de algo que los competidores nunca ofrecieron: clientes *nativos* verdaderamente multiplataforma (web, escritorio,

Android, iOS, además de HarmonyOS y Aurora OS), todos impulsados por un único contrato de backend. La clave es la propiedad sin concesiones. Un diseño HTTP/3 en todas partes, con microservicios totalmente desacoplados y cifrado SQLCipher AES-256 en reposo, lleva el rendimiento y la seguridad reservados hasta ahora a los SaaS propietarios a un sistema que alojas tú mismo: sin licencias por usuario, sin dependencia de proveedores, sin que tus datos abandonen tu infraestructura. Los equipos obtienen la experiencia JIRA-plus-Confluence que ya conocen, en su propio hardware y bajo una única licencia de código abierto.

Qué hay de innovador

- **Unified action-based /do API:** un único punto final, un único sobre, con enrutamiento por acciones. Las nuevas capacidades llegan como nuevas acciones, no como nuevas URLs, reduciendo la superficie de ataque, el código del cliente y la carga de documentación a un único contrato que comparten todas las plataformas.
- **HTTP/3 QUIC como transporte *predeterminado* entre servicios:** redes modernas de baja latencia y resilientes a las desconexiones desde el primer día, no añadidas como parche posterior.
- **Motor de permisos intercambiable:** entre una implementación local en proceso y un servicio respaldado por HTTP, junto con servicios opcionales e implementables de forma independiente para autenticación, permisos y localización. El mismo modelo de autorización, ya ejecute un proceso o un clúster.
- **Aislamiento de datos multiespacio mediante el flag --space-root:** cada proyecto tiene su propia base de datos y almacenamiento de activos aislados, de modo que inquilinos y proyectos se separan en el límite de almacenamiento, no mediante filtros de consulta.
- **Cifrado SQLCipher AES-256 en reposo:** los datos sensibles de los proyectos están protegidos en disco de forma transparente y predeterminada.
- **Detección automática cliente-servidor mediante difusión UDP en redes locales:** un cliente encuentra Core sin configuración manual.
- **Documentos V2, una auténtica “alternativa a Confluence”:** con edición paralela bajo bloqueo optimista, detección de conflictos y historial completo de cambios. Documentos colaborativos reales que residen en el mismo backend que el rastreador.

Principales desafíos técnicos y cómo los resolvimos

- **Seis plataformas cliente, un backend, cero deriva en el contrato.** Mantener clientes para Web/Angular, Escritorio/Tauri, Android/Kotlin, iOS/Swift, HarmonyOS y Aurora suele implicar seis integraciones API divergentes que se desincronizan. Eliminamos ese riesgo haciendo del único punto final /do con enrutamiento por acciones y su sobre fijo el *único* contrato: todos los clientes lo implementan de forma idéntica. Además, añadimos detección de servicios mediante difusión UDP para que los clientes localicen Core en la red sin configurar endpoints manualmente.
- **Desacoplar servicios sin pagar un peaje en latencia.** Dividir autenticación, permisos y localización en servicios implementables de forma independiente suele añadir un salto de red por llamada. Adoptamos HTTP/3 QUIC en todas las llamadas entre servicios para mantener esos saltos rápidos y resilientes a las desconexiones, y permitimos que cada servicio se ejecute de forma autónoma —incluso deshabilitado por completo en configuraciones de prueba—, de modo que el desacoplamiento es una opción de despliegue, no un coste fijo.
- **Colaboración al nivel de Confluence sin caos de escrituras perdidas.** La edición multiautor en tiempo real invita a conflictos. Documentos V2 los resuelve con espacios/páginas/control de versiones bajo bloqueo optimista, detección explícita de conflictos, un historial completo de cambios al que recurrir y sincronización WebSocket en tiempo real: colaboración que mantiene la coherencia en lugar de sobrescribir ediciones en silencio.
- **Cifrado en reposo sin sacrificar rendimiento.** SQLCipher AES-256 protege los datos en disco, pero añade sobrecarga por consulta. Lo compensamos con caché multicapa (LRU en memoria frente a Redis en el servicio de localización) para que rutas críticas como las búsquedas multilingües sigan siendo rápidas mientras los datos permanecen cifrados.

Pila tecnológica

- **Go + Gin** — seleccionados por su alto rendimiento y baja latencia en servicios HTTP, con despliegue en un único binario; incluyen el REST API de Core, su middleware JWT/CORS y el enrutador de acciones /do que actúa como fachada de todo el sistema.
- **HTTP/3 QUIC** — elegido como protocolo de transporte entre Core y sus servicios de Autenticación/Permisos/Localización, ya que el diseño multiplexado y de migración de conexiones de QUIC reduce la latencia en cola y resiste enlaces inestables donde TCP se bloquea.

- **PostgreSQL (prod) / SQLite (dev)** — un único modelo relacional que respalda el esquema de seguimiento y documentos en ambos motores: SQLite permite un desarrollo local sin configuración y basado en archivos, mientras que PostgreSQL asume el control en producción mediante un perfil production dedicado en Compose.
- **SQLCipher (AES-256)** — seleccionado para el cifrado transparente a nivel de base de datos en reposo, de modo que la protección de datos sensibles del proyecto no requiere criptografía en la capa de aplicación ni modificaciones en la escritura de consultas.
- **Redis** — elegido como capa de caché compartida detrás de una LRU en memoria en el servicio de Localización, proporcionando un caché de dos niveles que mantiene rápidas las búsquedas multilingües frecuentes incluso con la sobrecarga del cifrado subyacente.
- **Uber Zap + Lumberjack** — seleccionados para el registro estructurado y de bajo consumo de memoria, con rotación integrada, lo que garantiza que Core siga siendo observable en producción sin un crecimiento ilimitado de registros.
- **golang-jwt / JWT** — elegidos como mecanismo de autenticación sin estado; el token firmado viaja en el campo jwt de cada sobre /do, lo que asegura una autenticación uniforme en todos los clientes.
- **Angular 19 (+ Material, RxJS)** — seleccionado para un cliente web reactivo y basado en componentes, con un sistema de diseño Material maduro listo para usar.
- **Tauri 2.0 + Rust + Angular** — elegidos para distribuir una interfaz de escritorio nativa con una huella mínima, reutilizando la UI de Angular dentro de un webview respaldado por Rust en lugar de empaquetar un entorno de navegador completo.
- **Kotlin (Android) / Swift + SwiftUI (iOS)** — seleccionados para que los usuarios móviles obtengan clientes verdaderamente nativos y adaptados a cada plataforma, en lugar de una vista web encapsulada.
- **Docker / Docker Compose (compatible con Podman)** — elegidos para un despliegue reproducible y contenerizado, con comprobaciones de /health integradas y compatibilidad con Podman, evitando así la dependencia de un daemon o proveedor específico.
- **Testify (Go); Cypress/Playwright/Karma+Jasmine (clientes)** — seleccionados para pruebas automatizadas en capas que cubren tanto el contrato del backend como las interfaces de usuario de los clientes de forma independiente, alineándose con la arquitectura de un único backend y múltiples clientes.

Estado y notas de transparencia

- **Estado: beta.** HelixTrack Core es un microservicio REST API funcional; la extensión **Documentos V2** está documentada como aproximadamente un 95 % completa, con un problema conocido de mapeo de campos en la base de datos, por lo que no se presenta como totalmente implementada.
- **Licencia: por definir.** CLAUDE.md indica MIT, pero el archivo core/LICENSE de referencia es Apache 2.0; esta discrepancia debe resolverse antes de establecer la licencia de forma definitiva.
- Las cifras de rendimiento mencionadas en el README del proyecto (por ejemplo, más de 50 000 solicitudes/segundo, tiempos de consulta inferiores al milisegundo) son objetivos de diseño y marketing, no benchmarks publicados de forma independiente, por lo que se omiten en las afirmaciones anteriores.
- Las aplicaciones cliente (Web, Escritorio, Android, iOS, Aurora, HarmonyOS) residen en repositorios **privados** y solo se describen a nivel de producto.

Nivel de prioridad: Helix-primary y buque insignia de la línea de productos Helix-Track — clasificado por encima de cualquier proyecto Server Factory.